Offline field-change queue and retry-safe synchronization

Source: mobile/src/services/sync-queue.ts **Lines:** 83-156 **Revision:** a84fe8ffa1d4

```
  async enqueue(
    input: Omit<PendingMutation, "id" | "createdAt" | "attempts"> & {
      id?: string;
    },
  ) {
    if (!owner) throw new Error("Sign in before saving field records.");
    const mutation: PendingMutation = {
      ...input,
      id: input.id || `${Date.now()}-${Math.random().toString(36).slice(2)}`,
      createdAt: new Date().toISOString(),
      attempts: 0,
    };
    // Preserve every ordered write: replacing PATCHes loses independent fields.
    await change(owner, (queue) =>
      queue.some((m) => m.id === mutation.id) ? queue : [...queue, mutation],
    );
    return mutation;
  },
  pending: () => read(),
  async flush() {
    if (running || !replay || !owner) return;
    running = true;
    const id = owner;
    try {
      const network = await NetInfo.fetch();
      connected =
        !!network.isConnected && network.isInternetReachable !== false;
      if (!connected) return;
      await notify();
      for (const mutation of await read(id)) {
        if (owner !== id) break;
        try {
          await replay(mutation);
          await change(id, (queue) =>
            queue.filter((m) => m.id !== mutation.id),
          );
          await AsyncStorage.setItem(
            `${key(id)}.last`,
            new Date().toISOString(),
          );
        } catch (error: any) {
          await change(id, (queue) =>
            queue.map((m) =>
              m.id === mutation.id
                ? {
                    ...m,
                    attempts: m.attempts + 1,
                    lastError: String(
                      error?.response?.data?.message || error.message,
                    ),
                  }
                : m,
            ),
          );
          // Keep dependent records behind the failed parent until corrected/retried.
          break;
        }
      }
      if (
        onSynced &&
        owner === id &&
        Date.now() - lastReported > REPORT_EVERY_MS &&
        (await read(id)).length === 0
      ) {
        lastReported = Date.now();
        await onSynced().catch(() => {
          lastReported = 0;
        });
      }
    } finally {
      running = false;
      await notify();
    }
  },
```

Farm verification evidence and atomic status update

Source: backend/src/farm-verifications/farm-verifications.service.ts **Lines:** 14-71 **Revision:** a84fe8ffa1d4

```
14    async verifyFarm(officerUserId: string, verifyFarmDto: VerifyFarmDto) {
15      const {
16        farmId,
17        neighborLeft,
18        neighborRight,
19        mamcosApprovalStatus,
20        photoProofUrl,
21        notes,
22      } = verifyFarmDto;
23
24      // Find the field officer profile
25      const fieldOfficer = await this.prisma.mamcosStaff.findFirst({
26        where: { userId: officerUserId, role: MamcosStaffRole.FIELD_OFFICER },
27      });
28
29      if (!fieldOfficer) {
30        throw new NotFoundException(
31          `Field Officer profile for user ID ${officerUserId} not found`,
32        );
33      }
34
35      // Verify farm exists
36      const farm = await this.prisma.farm.findUnique({
37        where: { id: farmId },
38      });
39
40      if (!farm) {
41        throw new NotFoundException(`Farm with ID ${farmId} not found`);
42      }
43
44      if (farm.isVerified) {
45        throw new ConflictException(`Farm with ID ${farmId} is already verified`);
46      }
47
48      // Perform transaction to add verification record and update farm status
49      return this.prisma.$transaction(async (prisma) => {
50        const verification = await prisma.farmVerification.create({
51          data: {
52            farmId,
53            fieldOfficerId: fieldOfficer.id,
54            gpsVerified: true,
55            verifiedAt: new Date(),
56            neighborLeft,
57            neighborRight,
58            mamcosApproved: mamcosApprovalStatus,
59            photoProofUrl,
60            notes,
61          },
62        });
63
64        await prisma.farm.update({
65          where: { id: farmId },
66          data: { isVerified: true },
67        });
68
69        return verification;
70      });
71    }
```

Field survey model for soil, water access and GPS

Source: backend/prisma/schema.prisma **Lines:** 2642-2675 **Revision:** a84fe8ffa1d4

```
2642  model FarmFieldSurvey {
2643    id           String         @id @default(uuid())
2644    farmId       String         @map("farm_id")
2645    surveyedById String?        @map("surveyed_by_id")
2646    surveyDate   DateTime       @default(now()) @map("survey_date")
2647    source       FarmDataSource @default(FIELD_OFFICER)
2648
2649    // Soil
2650    soilPh            Float?  @map("soil_ph")
2651    soilTexture       String? @map("soil_texture")
2652    soilOrganicMatter Float?  @map("soil_organic_matter") // %
2653    soilNotes         String? @map("soil_notes")
2654
2655    // Road & water access
2656    roadDistanceMeters  Int?    @map("road_distance_meters")
2657    roadAccessQuality   String? @map("road_access_quality") // GOOD | FAIR | POOR
2658    waterSource         String? @map("water_source")
2659    waterDistanceMeters Int?    @map("water_distance_meters")
2660    waterReliability    String? @map("water_reliability")
2661
2662    // Physical
2663    slope        String?
2664    floodRisk    String? @map("flood_risk")
2665    observations String?
2666    latitude     Float?
2667    longitude    Float?
2668
2669    createdAt DateTime @default(now()) @map("created_at")
2670
2671    farm Farm @relation(fields: [farmId], references: [id], onDelete: Cascade)
2672
2673    @@index([farmId])
2674    @@map("farm_field_surveys")
2675  }
```